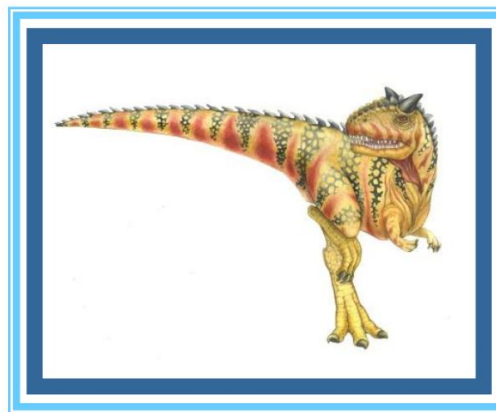


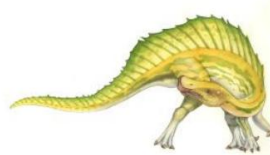
الفصل 6: جدولة وحدة المعالجة المركزية





الفصل 6: جدولة وحدة المعالجة المركزية

- المفاهيم الأساسية
- معايير الجدولة
- خوارزميات الجدولة
- جدولة المواضيع
- جدولة المعالجات المتعددة
- جدولة وحدة المعالجة المركزية في الوقت الفعلي
- أمثلة على أنظمة التشغيل
- تقييم الخوارزمية





أهداف

- لتقديم جدول وحدة المعالجة المركزية، والتي تشكل الأساس لأنظمة التشغيل متعددة البرمجة
- لوصف خوارزميات جدول وحدة المعالجة المركزية المختلفة
- لمناقشة معايير التقييم لاختيار خوارزمية جدول وحدة المعالجة المركزية لنظام معين
- لفحص خوارزميات الجدولة لعدة أنظمة تشغيل



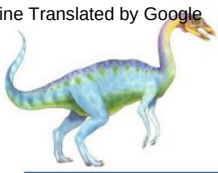
المفاهيم الأساسية

- فَمَ

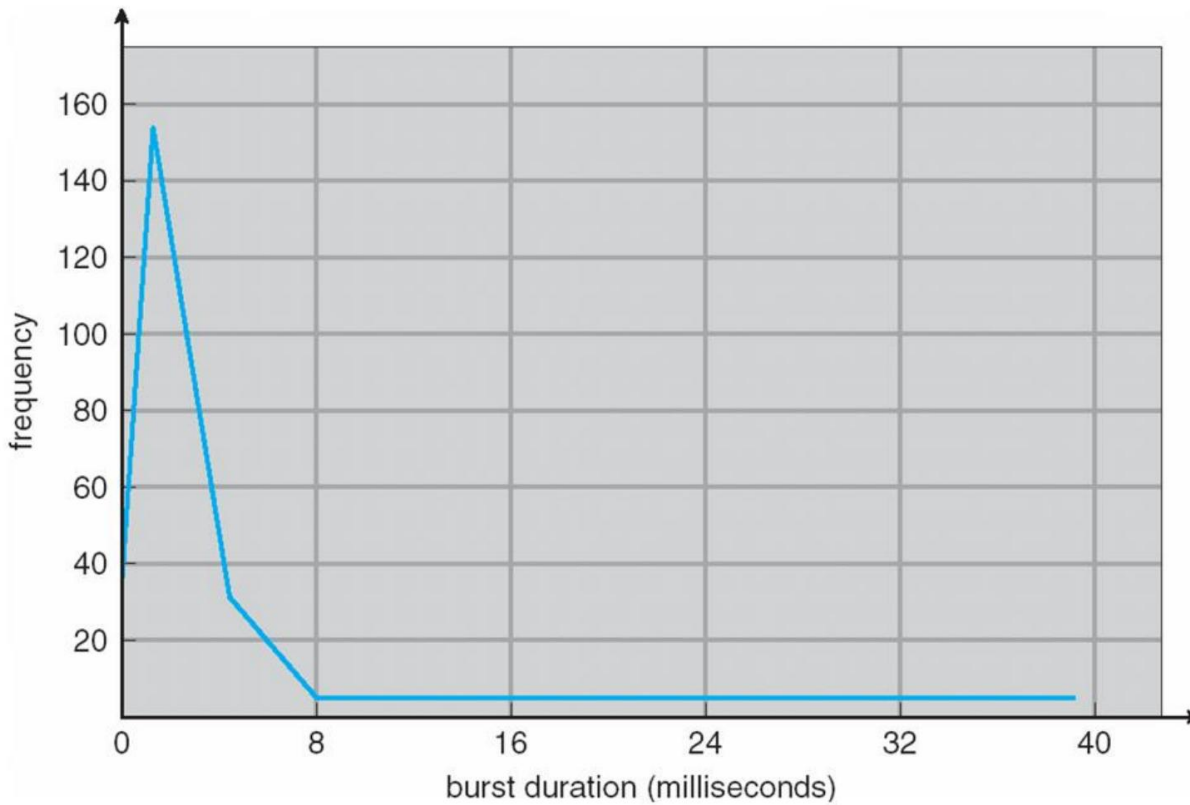
wait for I/O

} I/O burst





رسم بياني لأوقات انفجار وحدة المعالجة المركزية

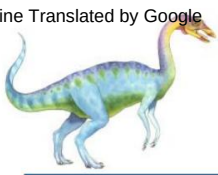




جدولة وحدة المعالجة المركزية

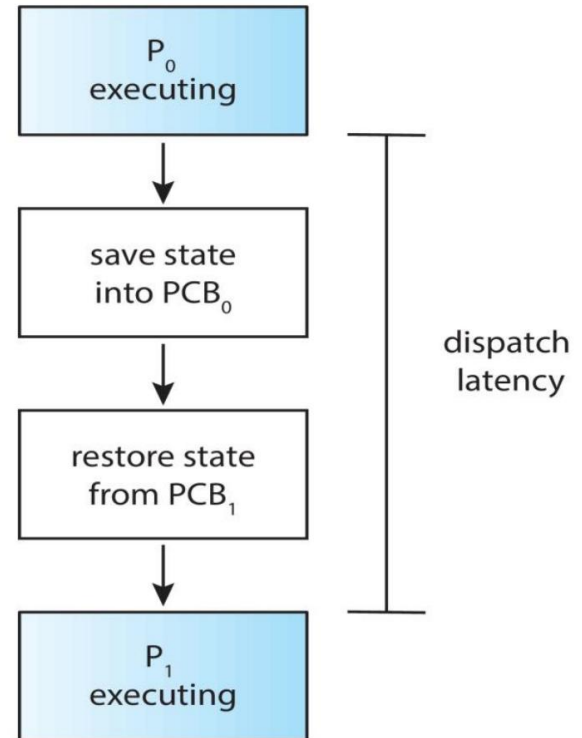
- يقوم **المجدول قصير المدى** بالاختيار من بين العمليات الموجودة في قائمة الانتظار الجاهزة، ويخصص وحدة المعالجة المركزية لواحدة منها
- يمكن ترتيب قائمة الانتظار بطرق مختلفة
- قد تحدث قرارات جدولة وحدة المعالجة المركزية عندما تكون العملية:
 1. التبديل من حالة التشغيل إلى حالة الانتظار
 2. التبديل من حالة التشغيل إلى حالة الاستعداد
 3. التبديل من حالة الانتظار إلى حالة الاستعداد
 4. ينهي
- الجدولة بموجب البندين 1 و 4 **غير استباقية**
- جميع الجداول الأخرى **استباقية**
 - خذ بعين الاعتبار الوصول إلى البيانات المشتركة
 - خذ بعين الاعتبار الأولوية أثناء وضع النواة
 - ضع في اعتبارك المقاطعات التي تحدث أثناء أنشطة نظام التشغيل المهمة

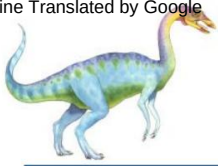




المرسل

- تمنح وحدة المرسل التحكم في وحدة المعالجة المركزية للعملية المحددة بواسطة الجدول قصير المدى؛ وهذا يتضمن:
 - تبديل السياق
 - التبديل إلى وضع المستخدم
 - الانتقال إلى الموقع الصحيح في برنامج المستخدم لإعادة تشغيل هذا البرنامج
- **زمن انتقال الإرسال** - الوقت الذي يستغرقه المرسل لإيقاف عملية واحدة وبدء تشغيل عملية أخرى

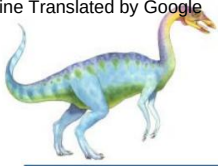




معايير الجدولة

- استخدام وحدة المعالجة المركزية - حافظ على وحدة المعالجة المركزية مشغولة قدر الإمكان
- معدل الإنتاج - عدد العمليات التي تكمل تنفيذها لكل وحدة زمنية
- وقت التنفيذ - مقدار الوقت اللازم لتنفيذ عملية معينة
- وقت الانتظار - مقدار الوقت الذي تنتظره العملية في قائمة الانتظار الجاهزة
- وقت الاستجابة - مقدار الوقت المستغرق من وقت تقديم الطلب حتى إنتاج الاستجابة الأولى، وليس الإخراج (لبيئة تقاسم الوقت)

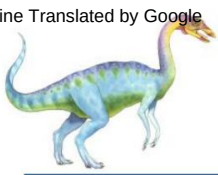




معايير تحسين خوارزمية الجدولة

- أقصى استخدام لوحدة المعالجة المركزية
- الحد الأقصى للإنتاجية
- الحد الأدنى لوقت التنفيذ
- الحد الأدنى لوقت الانتظار
- الحد الأدنى لوقت الاستجابة





جدولة أسبقية الحضور (FCFS)

عملية	وقت الانفجار
ص1	24
ص2	3
P3	3



ص1 ، ص2 ، P3 افترض أن العمليات تصل بالترتيب التالي:
مخطط جانت للجدول الزمني هو:



وقت الانتظار = 24



متوسط وقت الانتظار: $(0 + 24 + 27) / 3 = 17$





جدولة FCFS (تابع)

افترض أن العمليات تصل بالترتيب التالي:

P3 ، 1 ص ، 2 ص

□

مخطط جانث للجدول الزمني هو:



□

وقت الانتظار: $P_3 = 0$ ، $P_2 = 3$ ، $P_1 = 6$

□

متوسط وقت الانتظار: $(6 + 0 + 3) / 3 = 3$

□

أفضل بكثير من الحالة السابقة

□

تأثير القافلة - عملية قصيرة خلف عملية طويلة

□

ضع في اعتبارك عملية واحدة مرتبطة بوحدة المعالجة المركزية والعديد من العمليات المرتبطة بالإدخال والإخراج





جدولة أقصر وظيفة أولاً (SJF)

- ربط كل عملية بطول اندفاع وحدة المعالجة المركزية التالية
- استخدم هذه الأطوال لجدولة العملية بأقصر وقت
- SJF هو الحل الأمثل - حيث يوفر الحد الأدنى لمتوسط وقت الانتظار لمجموعة معينة من العمليات
- تكمن الصعوبة في معرفة طول طلب وحدة المعالجة المركزية التالي
- يمكن أن يسأل المستخدم

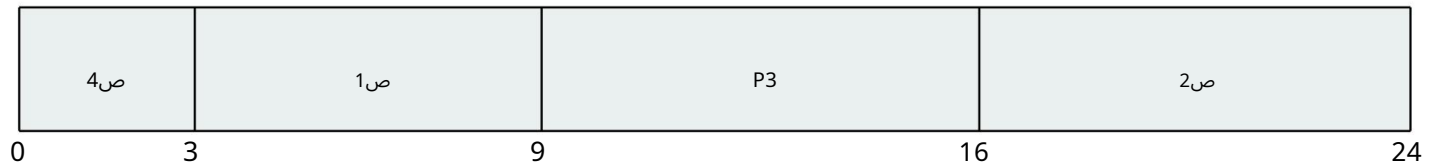




مثال على SJF

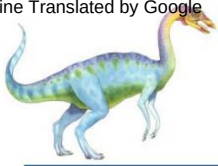
	وقت الانفجار
ص1	6
ص2	8
P3	7
ص4	3

مخطط جدول SJF



متوسط وقت الانتظار = $(3 + 16 + 9 + 0) / 4 = 7$



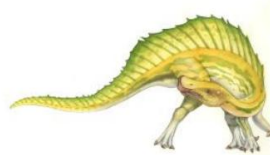


تحديد طول اندفاع وحدة المعالجة المركزية التالي

- يمكن تقدير الطول فقط - يجب أن يكون مشابهًا للطول السابق
- ثم اختر العملية ذات أقصر فترة زمنية متوقعة لاستهلاك وحدة المعالجة المركزية
- يمكن القيام بذلك باستخدام طول فترات وحدة المعالجة المركزية السابقة، باستخدام المتوسط الأسّي

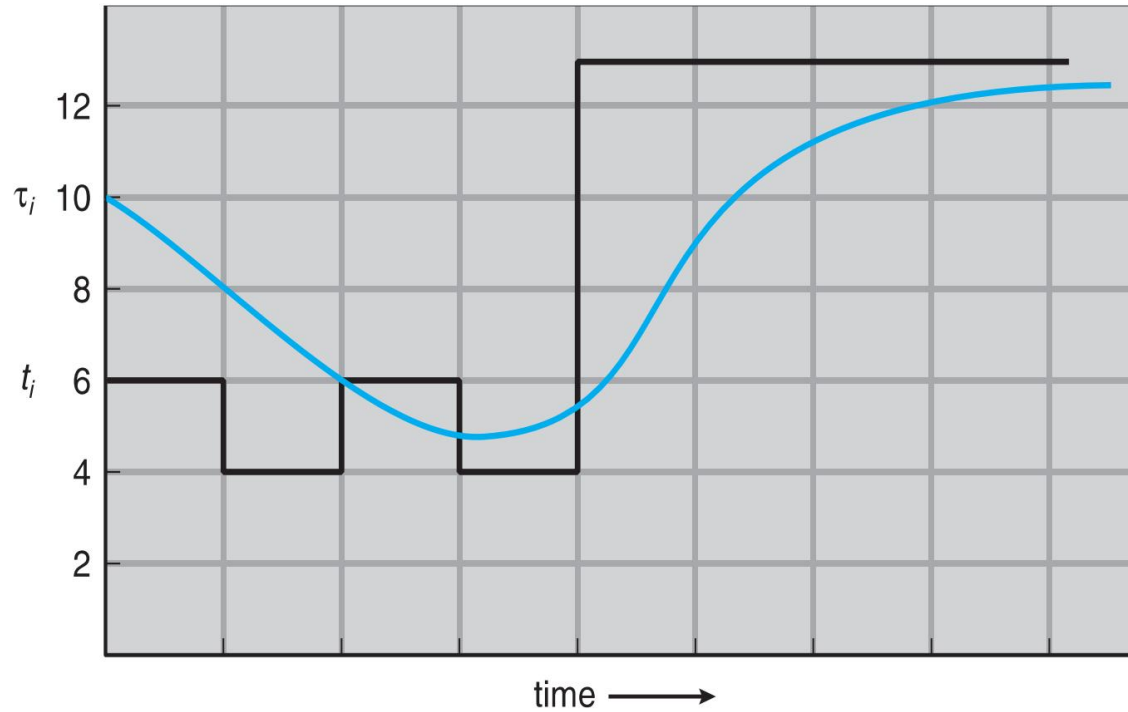
$$\begin{aligned}
 1. &= \text{الطول الفعلي لضغط وحدة المعالجة المركزية} \\
 2. &= \text{القيمة المتوقعة لنبضة وحدة المعالجة المركزية التالية} \\
 3. &= \text{القيمة المتوقعة لنبضة وحدة المعالجة المركزية التالية} \\
 4. &\text{عرف:} \quad \text{ت} = \frac{1}{n} + \frac{1}{n-1} + \dots + \frac{1}{2} + \frac{1}{1}
 \end{aligned}$$

- عادةً، يتم ضبط α على $\frac{1}{2}$
- النسخة الاستباقية تسمى **أقصر وقت متبقي أولاً**



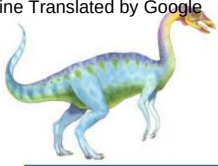


التنبؤ بطول فترة اندفاع وحدة المعالجة المركزية التالية



CPU burst (t_i)	6	4	6	4	13	13	13	...	
"guess" (τ_i)	10	8	6	6	5	9	11	12	...





أمثلة على المتوسط الأسي

$$\square = 0$$

$$\square_{n+1} = \square_n$$

التاريخ الحديث لا يهم

$$\square = 1$$

$$\square_{n+1} = \square_n$$

فقط آخر دفعة وحدة معالجة مركزية فعلية هي التي يتم حسابها

إذا قمنا بتوسيع الصيغة، نحصل على:

$$1 + \frac{1}{2^n} = \frac{1}{2^n} + 1$$

$$1 + \frac{1}{2^n} = \frac{1}{2^n} + 1$$

$$1 + \frac{1}{2^n} = \frac{1}{2^n} + 1$$

نظرًا لأن كلاً من $\square$ و $1 - \square$ أقل من أو يساوي 1، فإن كل حد لاحق له وزن أقل من سابقه





مثال على أقصر وقت متبقي أولاً



الآن نضيف مفاهيم أوقات الوصول المتنوعة والاستباق إلى التحليل

الوقت T	الوقت المتبقي	الوقت المتبقي
ص1	0	8
ص2	1	4
P3	2	9
ص4	3	5

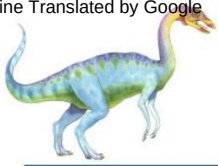


مخطط جانت SJF وقائي



متوسط وقت الانتظار = $[(10-1)+(1-1)+(17-2)+(5-3)]/4 = 26/4 = 6.5$ = مللي ثانية





جدولة الأولويات

- يرتبط رقم الأولوية (عدد صحيح) بكل عملية
- يتم تخصيص وحدة المعالجة المركزية للعملية ذات الأولوية الأعلى (أصغر عدد صحيح □ أعلى أولوية)
- وقائي
- غير استباقي
- SJF هو جدولة الأولوية حيث تكون الأولوية هي عكس وقت انفجار وحدة المعالجة المركزية التالي المتوقع
- المشكلة □ المجاعة - قد لا يتم تنفيذ العمليات ذات الأولوية المنخفضة أبدًا
- الحل □ الشيخوخة - مع تقدم الوقت تزداد أولوية العملية



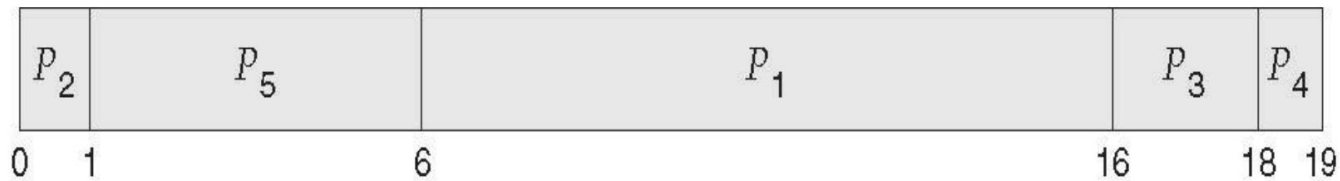


مثال على جدولة الأولويات

	عملية وقت	أولوية
ص1	10	3
ص2	1	1
P3	2	4
ص4	1	5
ص5	5	2

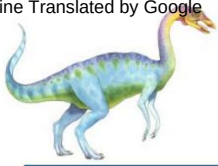


مخطط جانتي لجدولة الأولويات



متوسط وقت الانتظار 8.2 = مللي ثانية





جولة روبن (RR)

□ **تتمثل المهمة في معالجة وتوزيع طلبات في قائمة الانتظار بالترتيب الذي أتت عليه.** بعد انقضاء

□ العمليات في قائمة الانتظار الجاهزة. لا تزيد عن (1) عملية على 1/1 من وقت وحدة المعالجة
المركزية بوحدة زمنية دفعة واحدة. لا تنتظر في عملية وحدات زمنية أكثر.
س

س

□ يقوم المؤقت بمقاطعة كل كمية لتحديد موعد العملية التالية

□ أداء

□ كبير FIFO

□ صغير يجب أن يكون كبيرًا فيما يتعلق بتبديل السياق، وإلا فإن التكلفة الإجمالية ستكون مرتفعة للغاية س





مثال على RR مع $Time\ Quantum = 4$

عملية	وقت الانفجار
ص1	24
ص2	3
P3	3

مخطط جانت هو:

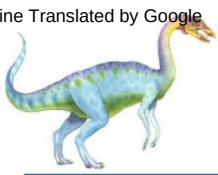


عادةً، يكون متوسط التحول أعلى من SJF، ولكن أفضل إجابة

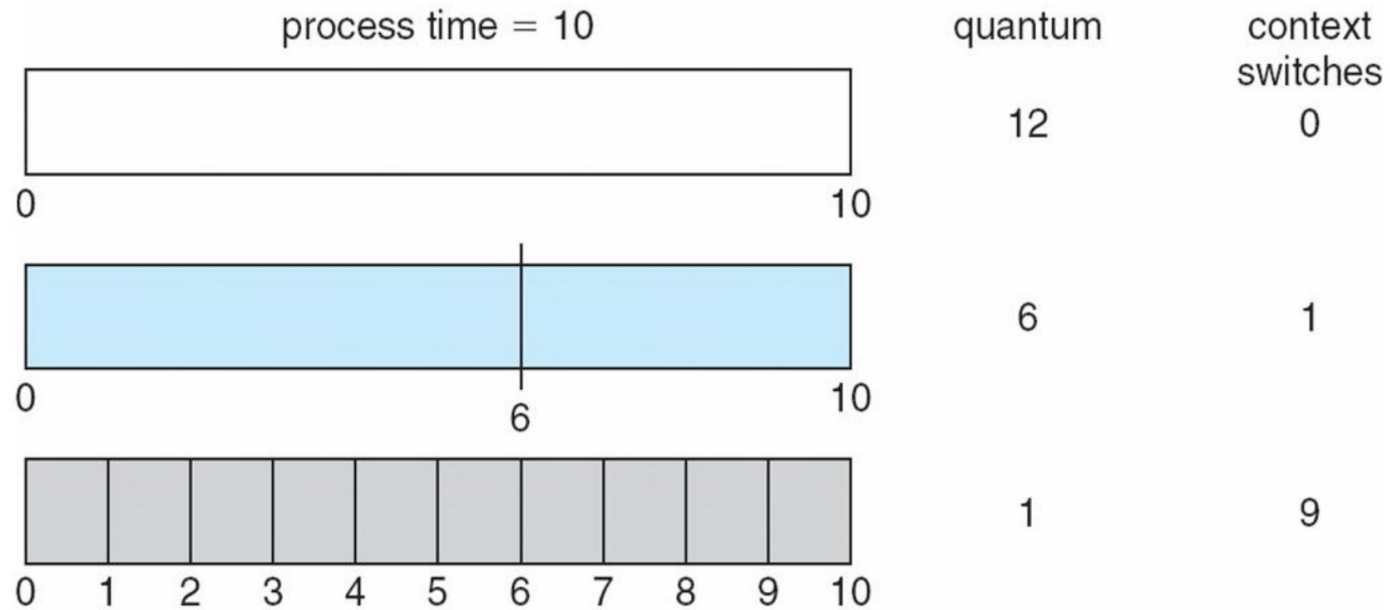
يجب أن يكون q كبيرًا مقارنة بوقت تبديل السياق

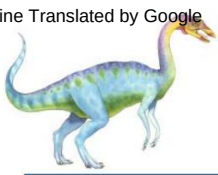
عادة ما يكون q من 10 مللي ثانية إلى 100 مللي ثانية، وتبديل السياق 10 < ثواني



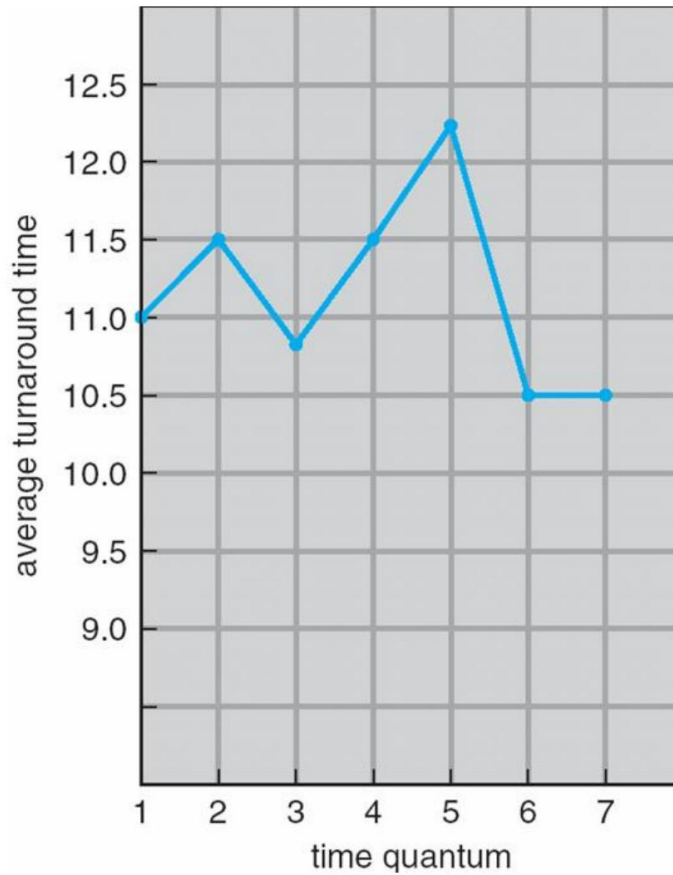


الوقت الكمي والتبديل السياقي





يختلف وقت التنفيذ باختلاف كمية الوقت



process	time
P_1	6
P_2	3
P_3	1
P_4	7

يجب أن تكون 80% من اندفاعات وحدة
المعالجة المركزية أقصر من q





قائمة انتظار متعددة المستويات

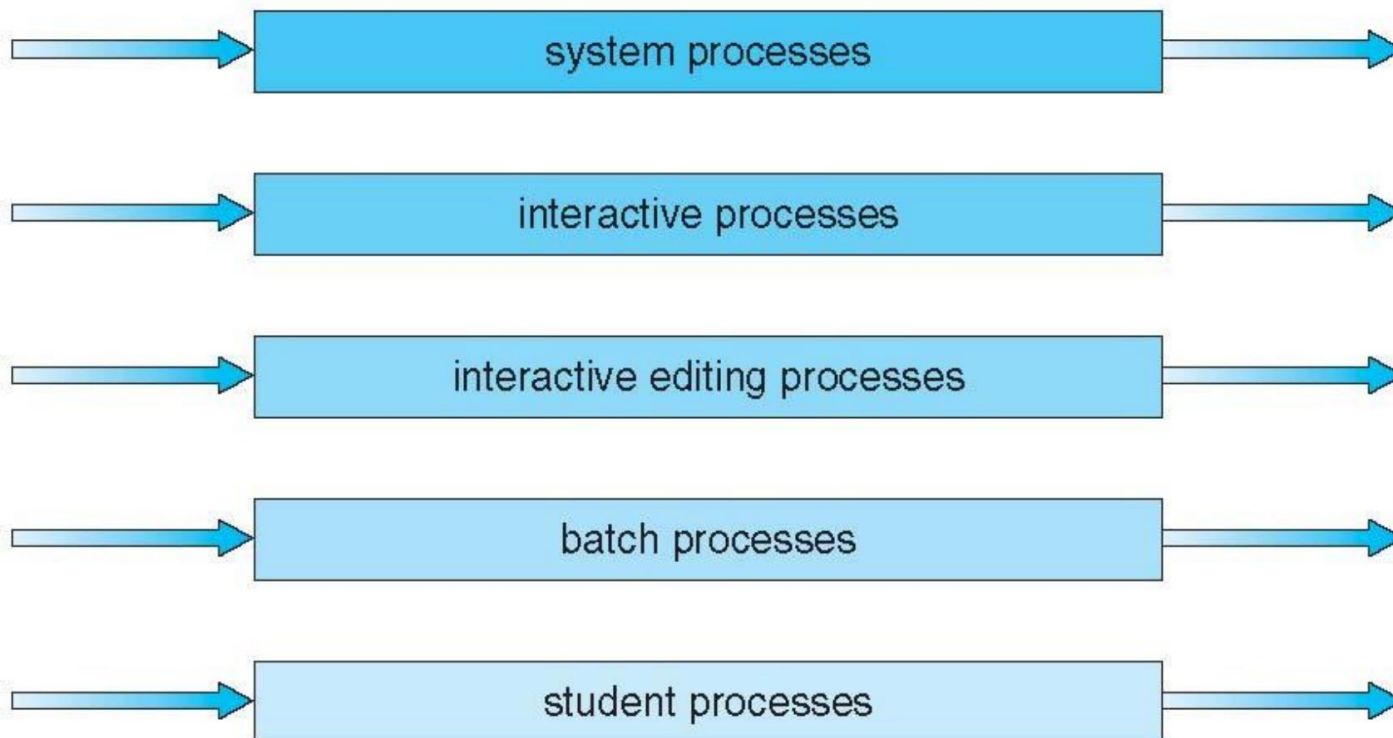
- يتم تقسيم قائمة الانتظار الجاهزة إلى قوائم انتظار منفصلة، على سبيل المثال:
 - المقدمة (تفاعلية)
 - الخلفية (الدفعة)
- عملية دائمة في قائمة انتظار معينة
- كل قائمة انتظار لديها خوارزمية جدولة خاصة بها:
 - المقدمة RR -
 - الخلفية FCFS -
- يجب أن يتم الجدولة بين الطوابير:
 - تم إصلاح جدولة الأولوية؛ (أي تقديم الخدمة من المقدمة ثم من الخلفية). احتمالية انقطاع الخدمة.
 - شريحة الوقت - يحصل كل طابور على مقدار معين من وقت وحدة المعالجة المركزية الذي يمكنه جدولته بين عملياته؛ أي 80% في المقدمة في RR
 - 20% إلى الخلفية في FCFS





جدولة قائمة انتظار متعددة المستويات

highest priority



lowest priority





قائمة انتظار الملاحظات متعددة المستويات

- يمكن أن تنتقل العملية بين الطوابير المختلفة؛ ويمكن تنفيذ الشيكوخة بهذه الطريقة
- يتم تحديد جدولة قائمة ردود الفعل متعددة المستويات بواسطة الملاحظات التالية:
 - عدد الطوابير
 - خوارزميات الجدولة لكل قائمة انتظار
 - الطريقة المستخدمة لتحديد موعد ترقية العملية
 - الطريقة المستخدمة لتحديد متى يتم تخفيض رتبة العملية
 - الطريقة المستخدمة لتحديد قائمة الانتظار التي ستدخلها العملية عندما تحتاج هذه العملية إلى الخدمة





مثال على قائمة ملاحظات متعددة المستويات

ثلاثة طوابير:

0 RR - مع زمن كمي قدره 8 ميلي ثانية

1 سزمن IRR الكمي 16 ميلي ثانية

2س - FCFS

الجدولة

وهو الذي وظيفة جديدة تدخل قائمة الانتظار التي يتم تقديمها ل FCFS

عند اكتساب وحدة المعالجة المركزية، تتلقى المهمة 8 مللي ثانية

إذا لم ينتهي في 8

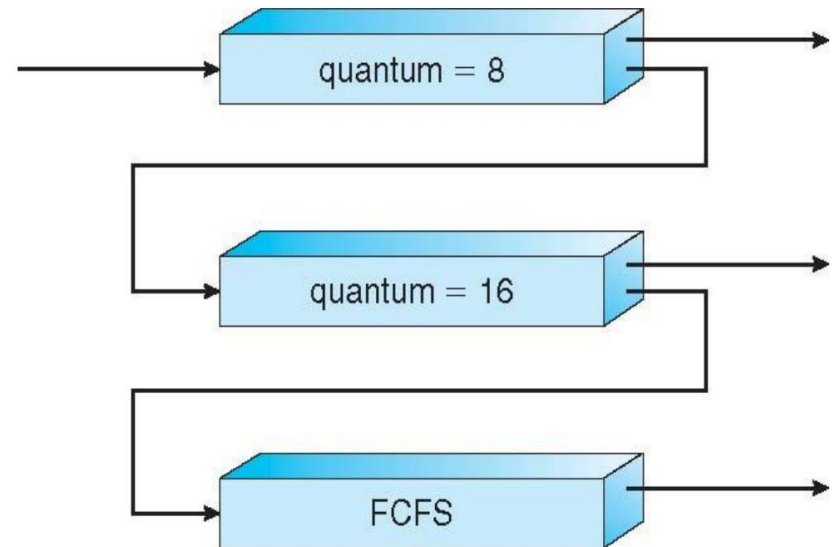
ميلي ثانية، يتم نقل المهمة إلى قائمة الانتظار

س

يتم تقديم FCFS مرة أخرى في الوظيفة ويتلقى 16 مللي ثانية إضافي

إذا لم يكتمل بعد، يتم استبقاؤه ونقله إلى

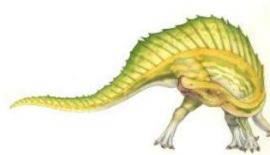
س قائمة الانتظار

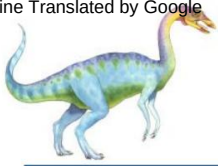




جدولة المواضيع

- التمييز بين الخيوط على مستوى المستخدم وخيوط مستوى النواة
- عندما يتم دعم الخيوط، يتم جدولة الخيوط، وليس العمليات
- نماذج من العديد إلى واحد ومن العديد إلى العديد، تقوم مكتبة الخيوط بجدولة خيوط على مستوى المستخدم لتشغيلها على LWP
- يُعرف باسم **نطاق التنافس على العملية (PCS)** نظرًا لأن المنافسة على الجدولة تقع ضمن العملية
- يتم ذلك عادةً من خلال تحديد الأولويات بواسطة المبرمج
- يتم جدولة خيط النواة على وحدة المعالجة المركزية المتاحة في **نطاق التنافس على النظام - (SCS)** المنافسة بين جميع الخيوط في النظام

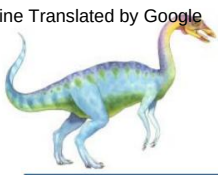




جدولة Pthread

- تسمح واجهة برمجة التطبيقات بتحديد PCS أو SCS أثناء إنشاء الخيط
- يقوم PTHREAD_SCOPE_PROCESS بجدولة الخيوط باستخدام جدول PCS
- يقوم PTHREAD_SCOPE_SYSTEM بجدولة الخيوط باستخدام جدول SCS
- يمكن تقييدها بنظام التشغيل Linux و OS X و macOS يسمحان فقط بنظام نطاق الخيوط





واجهة برمجة تطبيقات جدول Pthread

```
#تضمين <pthread.h>
#تضمين <stdio.h>

#تعريف NUM_THREADS 5

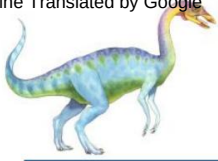
int main(int argc, char *argv[]) {
    int i;
    pthread_t tid[NUM_THREADS]; pthread_attr_t attr; /*
        الحصول على السمات الافتراضية */

    pthread_attr_init(&attr);

    /* الاستعلام أولاً عن النطاق الحالي */
    if (pthread_attr_getscope(&attr, &scope) != 0)
        fprintf(stderr, "غير قادر على الحصول على نطاق الجدولة\n");
    آخر {

        printf("PTHREAD_SCOPE_PROCESS"); إذا كان (النطاق
        PTHREAD_SCOPE_PROCESS) == وإذا كان (النطاق
        PTHREAD_SCOPE_SYSTEM) ==
        printf("PTHREAD_SCOPE_SYSTEM"); نظام)ftnirp
        آخر
        fprintf(stderr, "قيمة نطاق غير قانونية.\n");
    }
}
```





واجهة برمجة تطبيقات جدولة Pthread

*/تعيين خوارزمية الجدولة إلى PCS أو SCS */

pthread_attr_setscope(&attr, PTHREAD_SCOPE_SYSTEM);

*/إنشاء الخيوط */

ل (i = 0؛ i < NUM_THREADS؛ i++)

pthread_create(&tid[i], &attr, runner, NULL);

*/انضم الآن إلى كل موضوع */

ل (i = 0؛ i < NUM_THREADS؛ i++)

pthread_join(tid[i], NULL);

}

*/سيبدأ كل مؤشر ترابط بالتحكم في هذه الوظيفة */

void *runner(void *param)

{

*/قم ببعض العمل ... */

pthread_exit(0);

}





جدولة المعالجات المتعددة

□ تصبح جدولة وحدة المعالجة المركزية أكثر تعقيدًا عند توفر وحدات معالجة مركزية متعددة

□ **المعالجات المتجانسة** داخل المعالجات المتعددة

□ **المعالجة المتعددة غير المتماثلة** - معالج واحد فقط يمكنه الوصول إلى هياكل بيانات النظام، مما يخفف الحاجة إلى مشاركة البيانات

□ **المعالجة المتعددة المتماثلة - (SMP)** كل معالج يقوم بجدولة نفسه، وكل العمليات في قائمة انتظار جاهزة مشتركة، أو كل منها لديه قائمة انتظار خاصة به من العمليات الجاهزة

□ حاليًا، الأكثر شيوعًا

□ **تقارب المعالج** - تتمتع العملية بتقارب مع المعالج الذي تعمل عليه حاليًا

□ **تقارب ناعم**

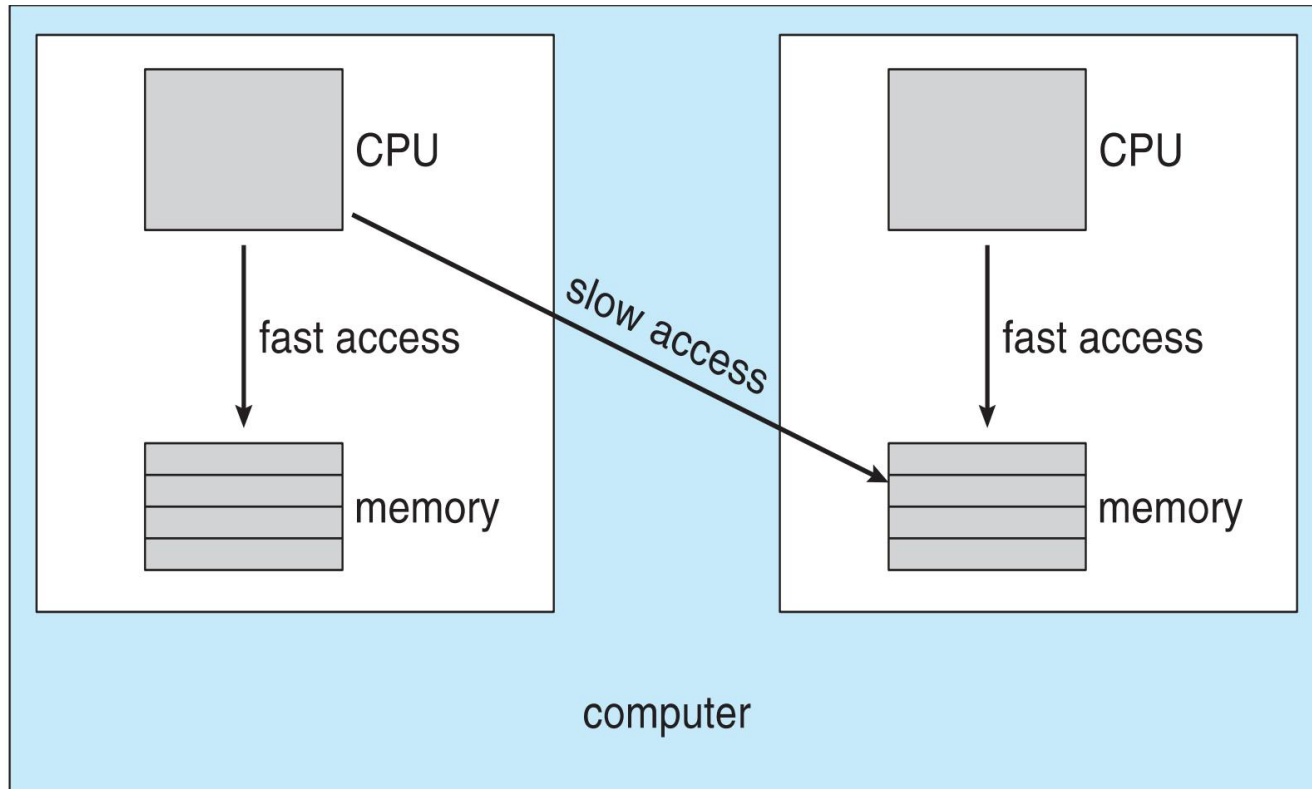
□ **تقارب صعب**

□ الاختلافات بما في ذلك مجموعات المعالج





NUMA وجدولة وحدة المعالجة المركزية



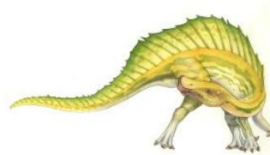
لاحظ أن خوارزميات وضع الذاكرة يمكنها أيضًا مراعاة التقارب





جدولة المعالجات المتعددة - موازنة التحميل

- إذا كان SMP، فمن الضروري إبقاء جميع وحدات المعالجة المركزية محملة لتحقيق الكفاءة
- تحاول موازنة التحميل الحفاظ على توزيع عبء العمل بالتساوي
- دفع الهجرة - فحص المهام بشكل دوري يتم على كل معالج، وإذا تم العثور عليها يتم دفع المهمة من وحدة المعالجة المركزية المحملة إلى وحدات المعالجة المركزية الأخرى
- سحب الهجرة - تقوم المعالجات الخاملة بسحب المهمة المنتظرة من المعالج المشغول





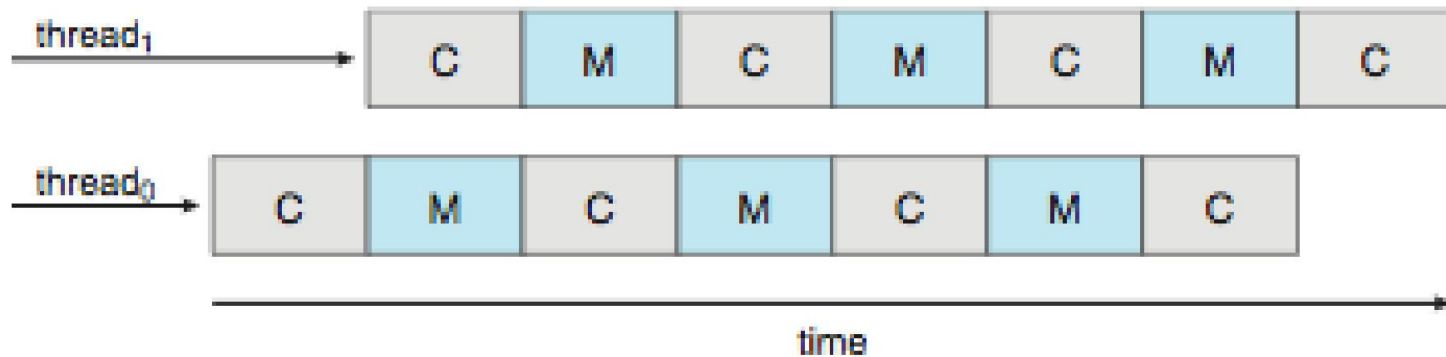
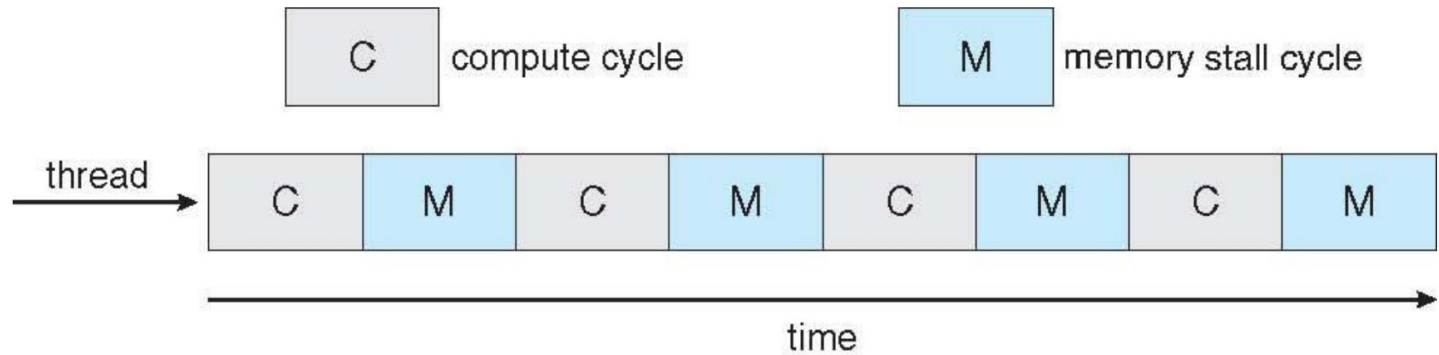
معالجات متعددة النواة

- الاتجاه الحديث هو وضع عدة أنوية للمعالج على نفس الشريحة المادية
- أسرع ويستهلك طاقة أقل
- تزايدت أيضًا الخيوط المتعددة لكل نواة
- يستفيد من توقف الذاكرة لإحراز تقدم في موضوع آخر أثناء استرجاع الذاكرة





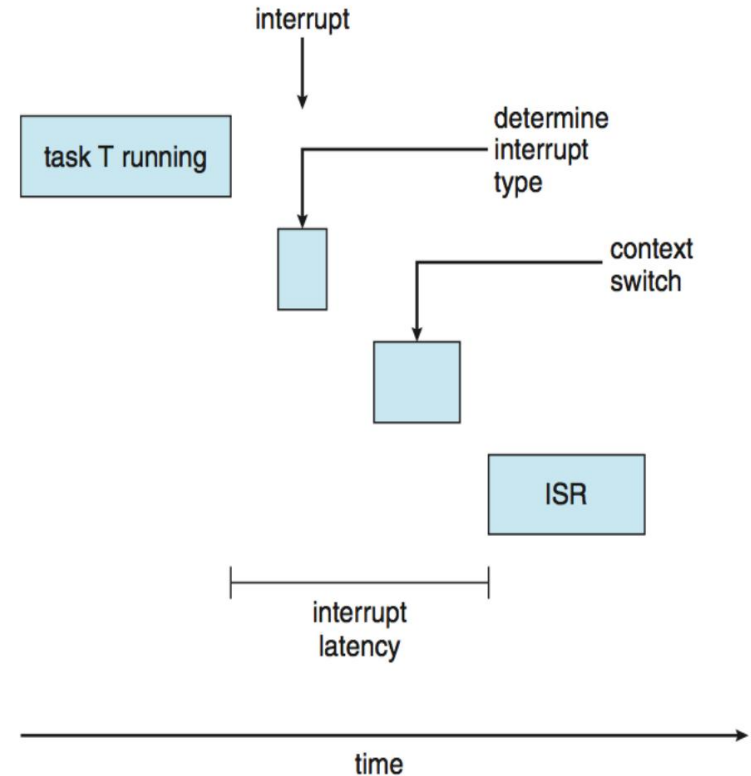
نظام متعدد الخيوط ومتعدد النواة

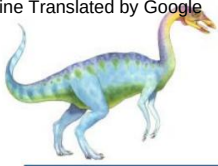




جدولة وحدة المعالجة المركزية في الوقت الفعلي

- يمكن أن يشكل تحديات واضحة
- أنظمة الوقت الحقيقي الناعمة - لا يوجد ضمان بشأن موعد جدولة العملية الحرجة في الوقت الحقيقي
- أنظمة الوقت الحقيقي الصعبة - يجب إنجاز المهمة في الموعد المحدد لها
- هناك نوعان من زمن الوصول يؤثران على الأداء
 1. زمن انتظار المقاطعة - الوقت من وصول المقاطعة إلى بدء الروتين الذي تقاطعه الخدمات
 2. زمن انتقال الإرسال - الوقت المخصص جدولة لإخراج العملية الحالية من وحدة المعالجة المركزية والتبديل إلى أخرى



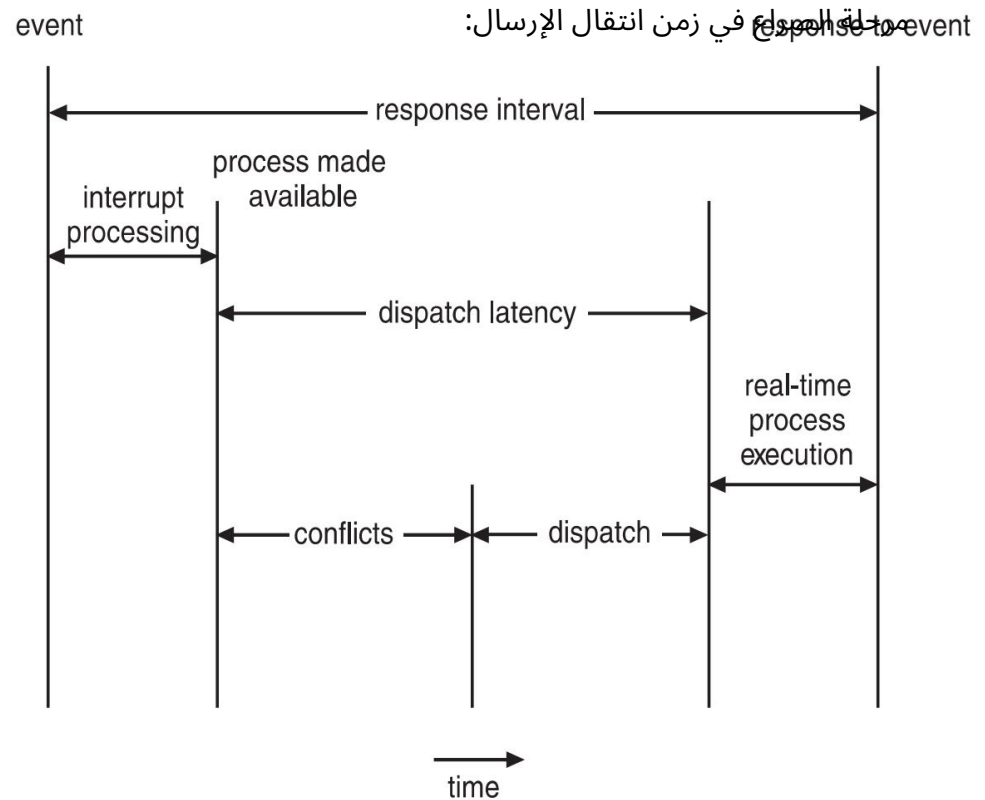


جدولة وحدة المعالجة المركزية في الوقت الفعلي (تابع)



1. منع أي عملية تعمل في وضع النواة

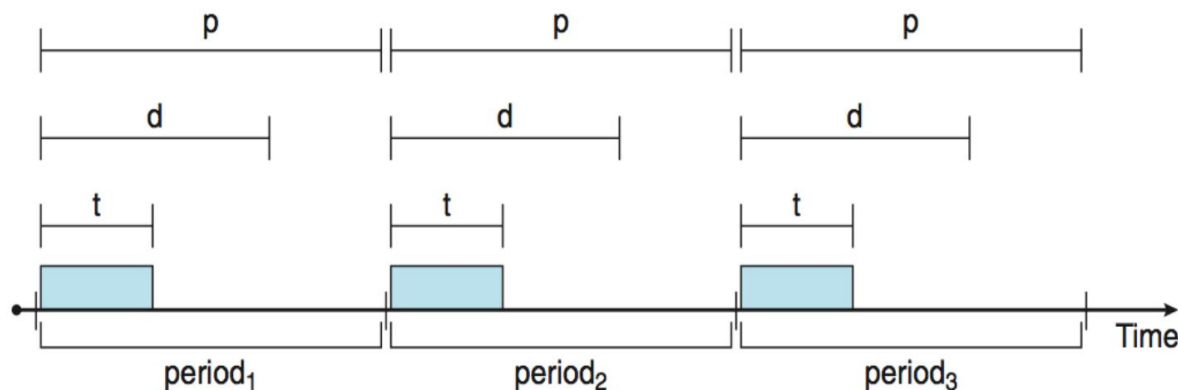
2. تحرير الموارد اللازمة للعمليات ذات الأولوية العالية من خلال عملية ذات أولوية منخفضة

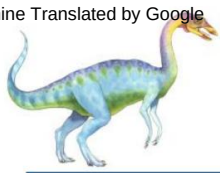




الجدولة القائمة على الأولوية

- بالنسبة للجدولة في الوقت الفعلي، يجب أن يدعم المُجدول الجدولة الاستباقية القائمة على الأولوية
- لكنها تضمن فقط الوقت الحقيقي الناعم
- بالنسبة للوقت الحقيقي الصعب يجب أيضاً توفير القدرة على تلبية المواعيد النهائية
- تمتلك العمليات خصائص جديدة: تتطلب العمليات **الدورية** وحدة المعالجة المركزية على فترات زمنية ثابتة
- **قوة التسليم** المعالجة ،
- **الوقت** 0
- **معدل** المهمة الدورية هو $1/$





المحاكاة الافتراضية والجدولة

□ يقوم برنامج المحاكاة الافتراضية بجدولة ضيوف متعددين على

وحدة المعالجة المركزية (CPUs)

□ كل ضيف يقوم بالجدول الزمني الخاص به

□ عدم معرفة أنه لا يمتلك وحدات المعالجة المركزية

□ قد يؤدي إلى ضعف وقت الاستجابة

□ يمكن أن يؤثر على ساعات اليوم للضيف

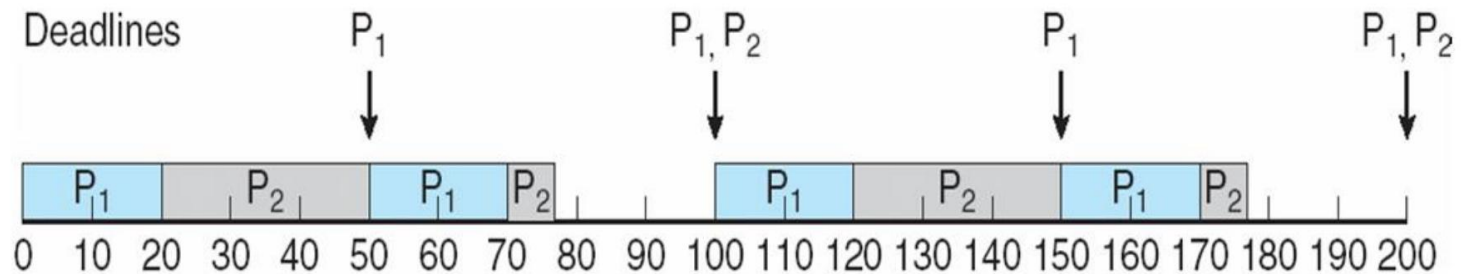
□ يمكن التراجع عن جهود خوارزمية الجدولة الجيدة للضيف

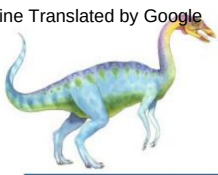




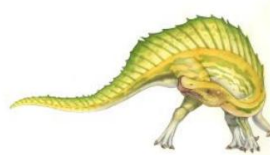
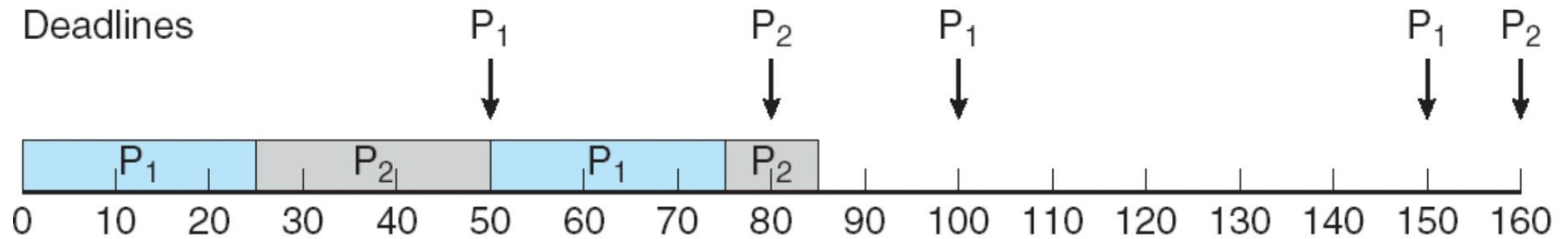
معدل الجدولة المونتونية

- يتم تحديد الأولوية بناءً على معكوس فترتها
- فترات أقصر = أولوية أعلى؛
- فترات أطول = أولوية أقل
- يتم تعيين أولوية أعلى لـ P_1 من P_2 .





المواعيد النهائية الفائتة مع جدولة معدل رتيب



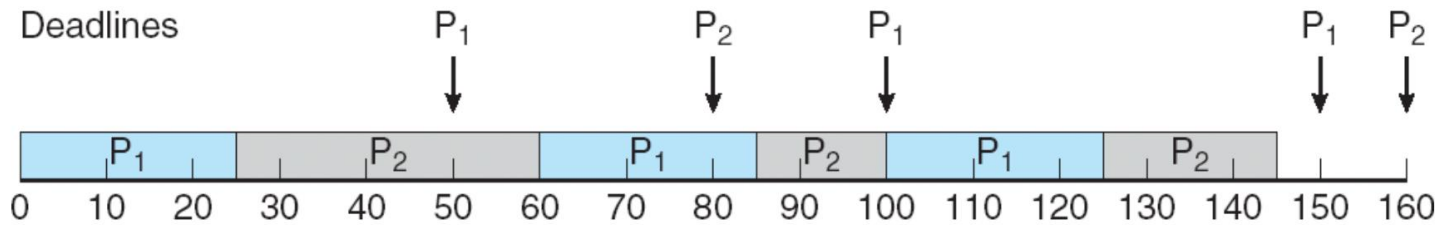


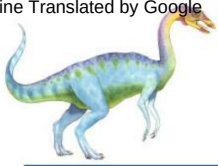
أقرب موعد نهائي للجدولة الأولى (EDF)



يتم توزيع الأولويات حسب المواعيد النهائية:

كلما كان الموعد النهائي مبكرًا، كلما زادت الأولوية؛ كلما كان الموعد النهائي متأخرًا، كلما انخفضت الأولوية





جدولة الحصص النسبية



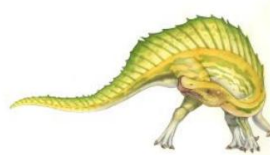
يتم تخصيص الأسهم بين جميع العمليات في النظام



ت < الأسهم استخدام الطلب



فن/المجموع الكلي
يضمن هذا أن كل تطبيق سوف يتلقى وقت المعالجة





جدولة POSIX في الوقت الفعلي

POSIX.1b معيار n

توفر واجهة برمجة التطبيقات n وظائف لإدارة الخيوط في الوقت الفعلي

n يحدد فئتين للجدولة لخيوط الوقت الحقيقي:

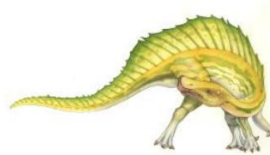
- **SCHED_FIFO**. اتم جدول الخيوط باستخدام استراتيجية FCFS مع قائمة انتظار. FIFO لا يوجد تقسيم زمني للخيوط ذات الأولوية المتساوية.

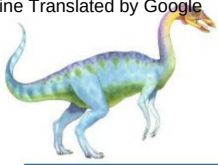
- **SCHED_RR**. 2. مشابه لـ SCHED_FIFO باستثناء حدوث تقسيم الوقت لـ خيوط ذات أولوية متساوية

n يحدد وظيفتين للحصول على سياسة الجدولة وتعيينها:

1. `pthread_attr_getsched_policy(pthread_attr_t *attr, int *policy)`

2. `pthread_attr_setsched_policy(pthread_attr_t *attr, int policy)` (السياسة)





واجهة برمجة تطبيقات الجدولة في الوقت الفعلي POSIX

```
#تضمين <pthread.h>

#تضمين <stdio.h>

#تعريف NUM_THREADS 5

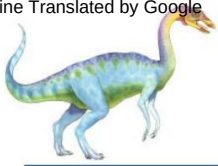
int main(int argc, char *argv[])
{
    int i;
    معرف مؤشر الترابط [عدد الخيوط];
    pthread_attr_t attr; /*
        الحصول على السمات الافتراضية */

    pthread_attr_init(&attr); /*
        الحصول على سياسة الجدولة الحالية */

    إذا (pthread_attr_getschedpolicy(&attr, &policy) != 0)
        fprintf(stderr, "غير قادر على الحصول على السياسة.\n");

    آخر {
        إذا (السياسة == SCHED_OTHER) printf("SCHED_OTHER\n");
        إذا (السياسة == SCHED_FIFO) printf("SCHED_FIFO\n");
        إذا (السياسة == SCHED_OTHER) printf("SCHED_OTHER\n");
    }
}
```





واجهة برمجة تطبيقات الجدولة في الوقت الفعلي POSIX (تابع)

```
/* تعيين سياسة الجدولة FIFO- أو RR أو (pthread_attr_setschedpolicy(&attr, SCHED_FIFO) != 0) fprintf(stderr, "تعيين سياسة الجدولة FIFO- أو RR أو (pthread_attr_setschedpolicy(&attr, SCHED_FIFO) != 0) fprintf(stderr, "OTHER" غير قادر على تعيين السياسة.\n"); if
```

```

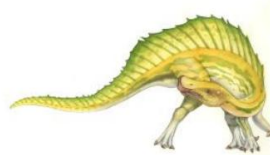
/* إنشاء الخيوط */
for (i = 0; i < NUM_THREADS; i++) {
    pthread_create(&tid[i], &attr, runner, NULL);

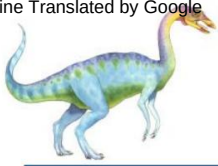
    /* انضم الآن إلى كل موضوع */
    for (i = 0; i < NUM_THREADS; i++) {
        pthread_join(tid[i], NULL);
    }

    /* سيبدأ كل مؤشر ترابط بالتحكم في هذه الوظيفة */
    void *runner(void *param)
    {
        /* قم ببعض العمل ... */

        pthread_exit(0);
    }
}

```





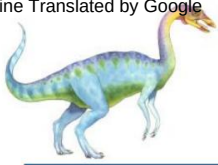
أمثلة على أنظمة التشغيل

جدولة لينكس

n جدولة Windows

جدولة سولاريس





جدولة Linux من خلال الإصدار 2.5

قبل إصدار ، kernel 2.5 تم تشغيل تنوع خوارزمية جدولة UNIX القياسية

الجدولة الوقت إلى الترتيب الثابت

ل استباقي، قائم على الأولوية

انطاقان للأولوية: تقاسم الوقت والوقت الحقيقي

انطاق الوقت الحقيقي من 0 إلى 99 وقيمة لطيفة من 100 إلى 140

اقم بتخطيط الأولوية العالمية مع القيم الأقل عددًا التي تشير إلى الأولوية الأعلى أولوية

الأولوية الأعلى تحصل على أكبر

يمكن تشغيل المهمة طالما أن هناك وقتًا متبقيًا في شريحة الوقت (نشط)

إذا لم يتبق وقت (انتهت صلاحيته)، فلن تتمكن من تشغيله حتى تستخدم جميع المهام الأخرى شرائحها

ا يتم تتبع جميع المهام القابلة للتشغيل في بنية بيانات قائمة انتظار التشغيل لكل وحدة معالجة مركزية

مصفوفتان ذات أولوية (نشطة، منتهية الصلاحية)

المهام المفهرسة حسب الأولوية

عندما لا يكون هناك المزيد من النشاط، يتم تبادل المصفوفات

لقد عمل بشكل جيد، ولكن أوقات الاستجابة للعمليات التفاعلية كانت ضعيفة





جدولة Linux في الإصدار 2.6.23 وما فوق

ن

المجدول

ن جدولة الفصول الدراسية

لكل منهما أولوية محددة

يختار المجدول المهمة ذات الأولوية الأعلى في أعلى فئة **جدولة**

ل بدلاً من الكم المبني على تخصيصات زمنية ثابتة، بناءً على نسبة وحدة المعالجة المركزية

وقت

أتم تضمين 2 فصول دراسية مجدولة، ويمكن إضافة فصول أخرى

1. الافتراضي

2. في الوقت الحقيقي

n الكم المحسوب بناءً على **القيمة اللطيفة** من 20- إلى +19

القيمة الأقل هي الأولوية الأعلى

احسب زمن الوصول المستهدف - الفترة الزمنية التي يجب أن يتم خلالها تشغيل المهمة لمدة لا تقل عن

مرة واحدة

يمكن أن يزيد زمن الوصول **المستهدف** إذا زاد عدد المهام النشطة

يحافظ **مجدول CFS** على **وقت التشغيل الافتراضي** لكل مهمة في متغير **vruntime**

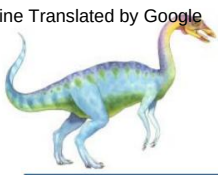
يرتبط بعامل الاضمحلال بناءً على أولوية المهمة - الأولوية الأقل هي الأعلى

معدل الاضمحلال

الأولوية الافتراضية العادية تعطي وقت تشغيل افتراضي = وقت تشغيل فعلي

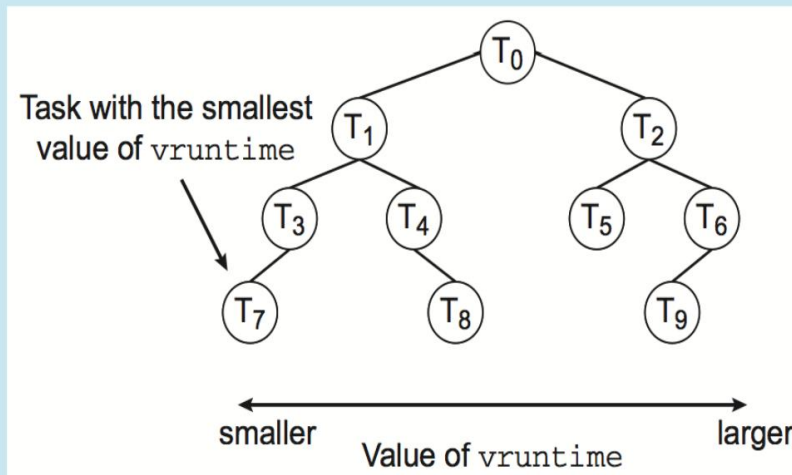
n لتحديد المهمة التالية التي سيتم تشغيلها، يختار المجدول المهمة ذات أقل وقت تشغيل افتراضي





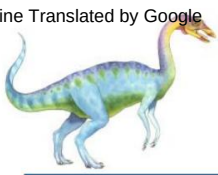
أداء CFS

The Linux CFS scheduler provides an efficient algorithm for selecting which task to run next. Each runnable task is placed in a red-black tree—a balanced binary search tree whose key is based on the value of `vruntime`. This tree is shown below:



When a task becomes runnable, it is added to the tree. If a task on the tree is not runnable (for example, if it is blocked while waiting for I/O), it is removed. Generally speaking, tasks that have been given less processing time (smaller values of `vruntime`) are toward the left side of the tree, and tasks that have been given more processing time are on the right side. According to the properties of a binary search tree, the leftmost node has the smallest key value, which for the sake of the CFS scheduler means that it is the task with the highest priority. Because the red-black tree is balanced, navigating it to discover the leftmost node will require $O(\lg N)$ operations (where N is the number of nodes in the tree). However, for efficiency reasons, the Linux scheduler caches this value in the variable `rb_leftmost`, and thus determining which task to run next requires only retrieving the cached value.





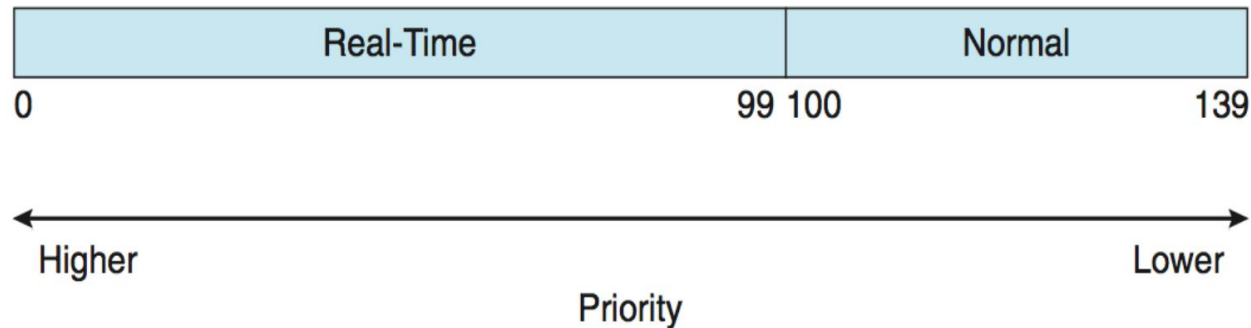
جدولة لينكس (تابع)

n الجدولة في الوقت الفعلي وفقاً لـ POSIX.1b المهام في الوقت الفعلي لها

أولويات ثابتة

n خريطة في الوقت الفعلي بالإضافة إلى الخريطة الطبيعية في مخطط الأولوية العالمية

قيمة لطيفة لـ -20 خريطة للأولوية العالمية n 100 قيمة لطيفة لـ +19 خريطة للأولوية 139





جدولة Windows

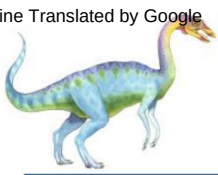
n يستخدم نظام التشغيل Windows جدولة استباقية تعتمد على الأولوية **n** يتم تشغيل مؤشر الترابط ذي الأولوية الأعلى بعد ذلك **n** يكون **المرسل** هو المجدول **n** يتم تشغيل مؤشر الترابط حتى (1) يتم حظره، (2) استخدام شريحة زمنية، (3) يتم استبقاؤه بواسطة مؤشر ترابط ذي أولوية أعلى **n** يمكن لمؤشرات الوقت الفعلي استباق مؤشرات الترابط غير الفعلية **n** مخطط أولوية من 32 مستوى **n** فئة المتغير هي 1-15، وفئة الوقت الفعلي هي 16-31

n الأولوية 0 هي مؤشر ترابط إدارة الذاكرة

n قائمة انتظار لكل أولوية **n** إذا لم يكن هناك مؤشر ترابط قابل للتشغيل،

يتم تشغيل **مؤشر ترابط خامل**





فئات أولوية Windows

n تحدد واجهة برمجة تطبيقات Win32 عدة فئات أولوية يمكن أن تنتمي إليها العملية

الفئة الأولوية في الوقت الفعلي، فئة الأولوية العالية،

أعلى من فئة الأولوية العادية، فئة الأولوية العادية،

فئة الأولوية أقل من العادية، فئة الأولوية الخاملة

جميعها متغيرة باستثناء الوقت الحقيقي

n يتمتع الخيط ضمن فئة أولوية معينة بأولوية نسبية

الوقت الحرج، الأعلى، أعلى من الطبيعي، الطبيعي، أقل من الطبيعي،

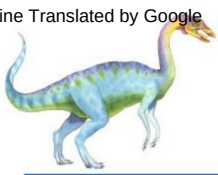
أدنى، حامل

يتم الجمع بين فئة الأولوية والأولوية النسبية لإعطاء أولوية رقمية

n الأولوية الأساسية هي NORMAL ضمن الفصل

n إذا انتهت صلاحية الكم، يتم تخفيض الأولوية، ولكن لا تقل أبدًا عن القاعدة





فئات أولوية Windows (تابع)

n إذا حدث الانتظار، يتم تعزيز الأولوية اعتمادًا على ما تم انتظاره

تم تعزيز أولوية نافذة المقدمة بمقدار 3 أضعاف

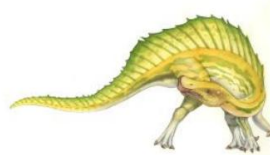
n أضاف نظام التشغيل Windows 7 **جدولة وضع المستخدم (UMS)** تعمل التطبيقات على إنشاء وإدارة مؤشرات

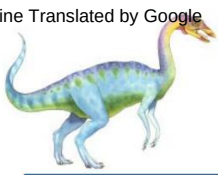
الترابط بشكل مستقل عن النواة

ل لعدد كبير من الخيوط، أكثر كفاءة بكثير

تأتي جداول UMS من مكتبات لغات البرمجة مثل

إطار عمل (ConcRT) **Concurrent Runtime** C++





أولويات Windows

	real-time	high	above normal	normal	below normal	idle priority
time-critical	31	15	15	15	15	15
highest	26	15	12	10	8	6
above normal	25	14	11	9	7	5
normal	24	13	10	8	6	4
below normal	23	12	9	7	5	3
lowest	22	11	8	6	4	2
idle	16	1	1	1	1	1





سولاريس

n الجدولة القائمة على الأولوية

n ستة فصول متاحة

ا تقاسم الوقت (افتراضي) ا (TS) تفاعلي ا

(IA) الوقت الحقيقي (RT)

نظام (SYS)

ل الحصة العادلة (FSS)

أ أولوية ثابتة (FP)

n يمكن أن يكون الخيط المعطى في فئة واحدة في كل مرة

n كل فئة لها خوارزمية جدولة خاصة بها n تقاسم الوقت عبارة عن قائمة

انتظار ملاحظات متعددة المستويات

ا جدول قابل للتحميل وقابل للتكوين بواسطة مسؤول النظام





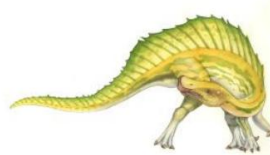
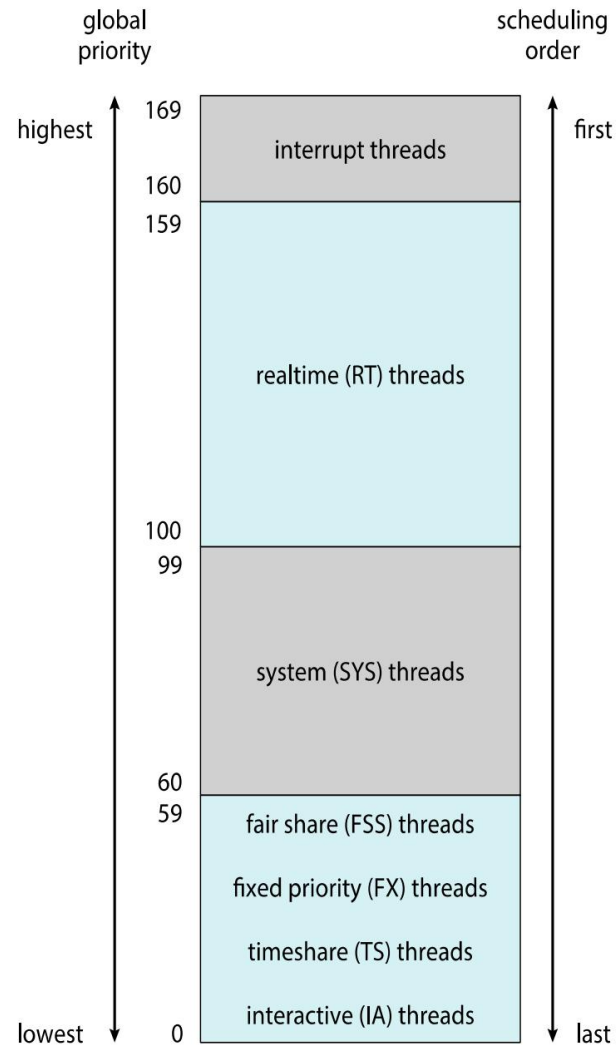
جدول توزيع سولاريس

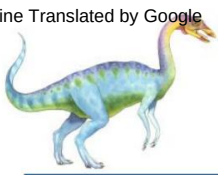
priority	time quantum	time quantum expired	return from sleep
0	200	0	50
5	200	0	50
10	160	0	51
15	160	5	51
20	120	10	52
25	120	15	52
30	80	20	53
35	80	25	54
40	40	30	55
45	40	35	56
50	40	40	58
55	40	45	58
59	20	49	59





جدولة سولاريس





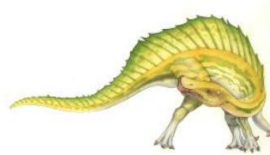
جدولة سولاريس (تابع)

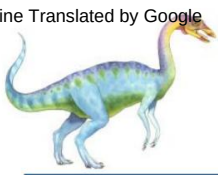
يقوم **المجدول** بتحويل الأولويات الخاصة بالفئة إلى أولويات عالمية لكل سلسلة أولوية

ا يتم تشغيل الخيط ذو الأولوية الأعلى بعد ذلك

ا يتم التشغيل حتى (1) يتم حظره، (2) يستخدم شريحة زمنية، (3) يتم إيقافه بواسطة خيط ذو أولوية أعلى

ا تم تحديد عدة خيوط بنفس الأولوية عبر RR





تقييم الخوارزمية

n كيفية اختيار خوارزمية جدولة وحدة المعالجة المركزية لنظام التشغيل؟

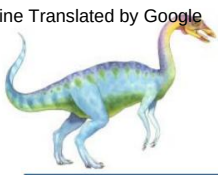
n تحديد المعايير، ثم تقييم الخوارزميات

n النمذجة الحتمية ل نوع التقييم التحليلي ل يأخذ عبء عمل محدد مسبقًا ويحدد أداء كل خوارزمية لذلك

الحمل n ضع في اعتبارك 5 عمليات تصل في الوقت: 0:

Process	Burst Time
P_1	10
P_2	29
P_3	3
P_4	7
P_5	12





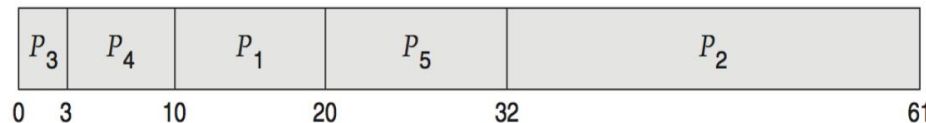
التقييم الحتمي

- لكل خوارزمية، احسب الحد الأدنى لمتوسط وقت الانتظار
- بسيطة وسريعة، ولكنها تتطلب أرقامًا دقيقة للإدخال، وتنطبق فقط على تلك المدخلات

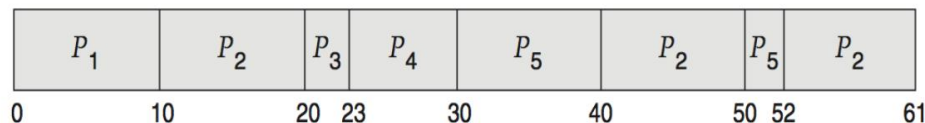
□ FCS هو 28 ملي ثانية:



□ SFJ غير الاستباقي هو 13 ملي ثانية:



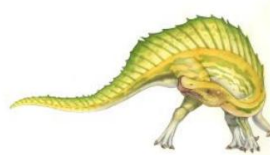
□ RR هو 23 ملي ثانية:

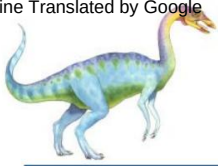




نماذج الانتظار

- يصف وصول العمليات، واندفاعات وحدة المعالجة المركزية والإدخال/الإخراج بشكل احتمالي
- عادةً ما تكون أسية، ويتم وصفها بالمتوسط
- يحسب متوسط الإنتاجية والاستخدام ووقت الانتظار وما إلى ذلك
- نظام الكمبيوتر هو عبارة عن شبكة من الخوادم، كل منها يحتوي على قائمة انتظار من العمليات المنتظرة
- معرفة معدلات الوصول ومعدلات الخدمة
- يحسب الاستخدام ومتوسط طول قائمة الانتظار ومتوسط وقت الانتظار وما إلى ذلك

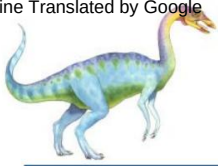




صيغة ليتل

- = متوسط طول قائمة الانتظار ن
 - = متوسط وقت في الطابور في انتظار
 - = متوسط معدل الوصول إلى قائمة الانتظار ل
 - قانون ليتل - في الحالة المستقرة، يجب أن تساوي العمليات التي تغادر قائمة الانتظار العمليات التي تصل إليها، وبالتالي:
- $$\bar{n} = \lambda \bar{w}$$
- صالحة لأي خوارزمية جدولة وتوزيع وصول
 - على سبيل المثال، إذا وصل في المتوسط 7 عمليات في الثانية، وعادةً ما يكون هناك 14 عملية في قائمة الانتظار، فإن متوسط وقت الانتظار لكل عملية 2 = ثانية





المحاكاة



نماذج الانتظار محدودة



المحاكاة أكثر دقة



نموذج مبرمج لنظام حاسوبي



الساعة متغيرة



جمع الإحصائيات التي تشير إلى أداء الخوارزمية



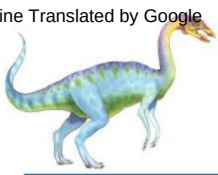
البيانات اللازمة لمحاكاة القيادة التي تم جمعها عبر

مولد أرقام عشوائية حسب الاحتمالات

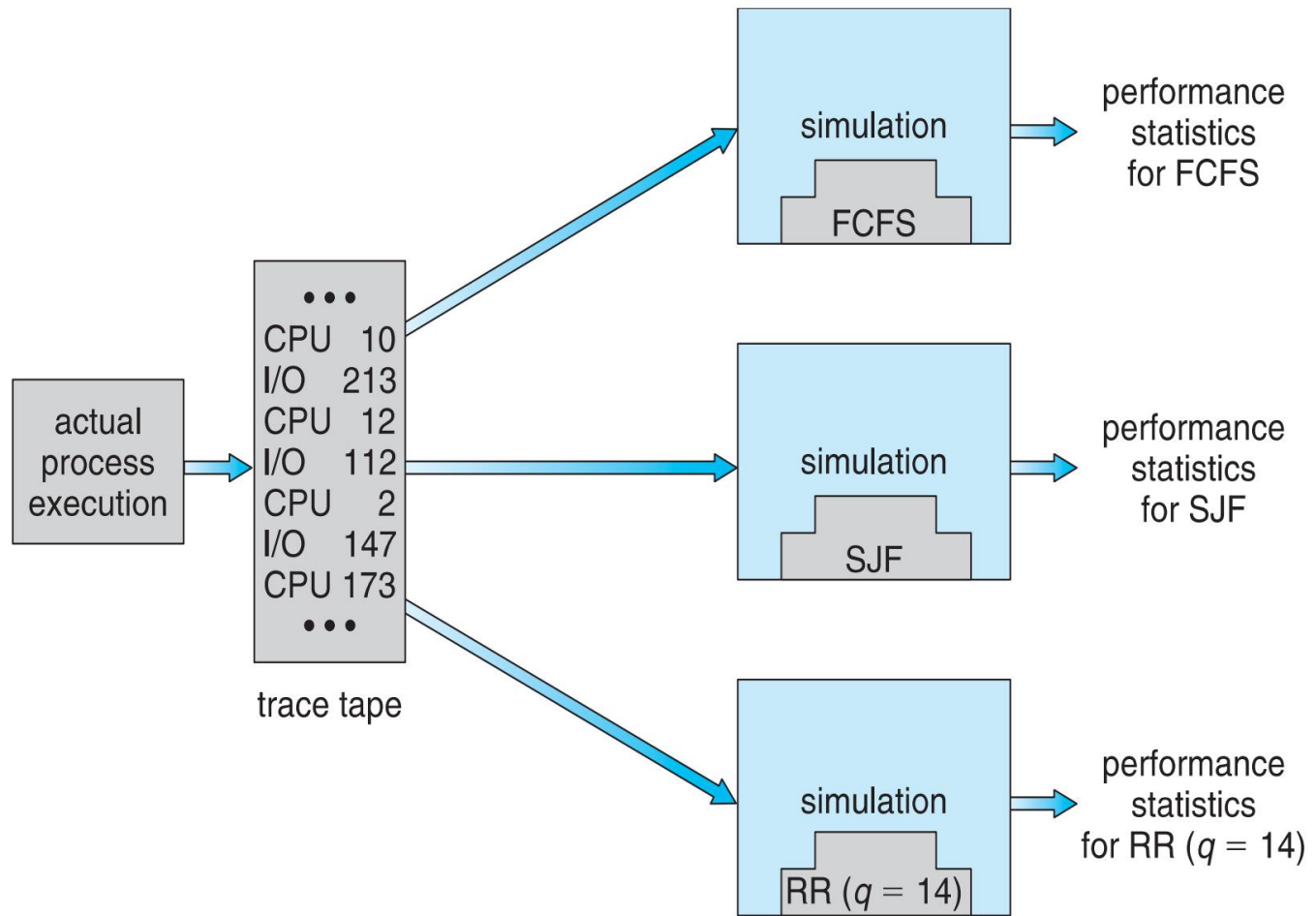
التوزيعات المحددة رياضياً أو تجريبياً تسجل أشرطة التتبع تسلسلات الأحداث الحقيقية في الأنظمة

الحقيقية





تقييم مُجدولات وحدة المعالجة المركزية عن طريق المحاكاة





تطبيق

- حتى المحاكاة لها دقة محدودة
- ما عليك سوى تنفيذ جدولة جديدة واختبارها في الأنظمة الحقيقية
 - تكلفة عالية ومخاطر عالية
 - تختلف البيئات
- يمكن تعديل معظم الجداول المرنة لكل موقع أو لكل نظام
- أو واجهات برمجة التطبيقات لتعديل الأولويات
- ولكن مرة أخرى تختلف البيئات



نهاية الفصل السادس

